

Теоретический материал

Одним из ключевых преимуществ ООП является возможность **повторного использования кода**. Это избавляет от дублирования и делает структуру программы логичной. Выделяют два основных типа отношений между объектами: **IS-A** («является») и **HAS-A** («содержит»).

1. Отношения IS-A (Наследование)

Принцип **IS-A** базируется на наследовании классов или реализации интерфейсов. Он указывает на иерархическую связь:

- **Наследование:** Если класс `HeavyBox` расширяет `Box`, мы говорим: «Тяжелый ящик **является** ящиком» (`HeavyBox IS-A Box`).
- **Интерфейсы:** Если класс `Transport` реализует интерфейс `Moveable`, это значит, что транспорт **является** движимым объектом.

Наследование идеально подходит для объектов с общим поведением. Однако это «жесткая» связь: изменяя родительский класс, мы автоматически влияем на все дочерние. Если же классы разные по сути, но должны работать вместе, используют другой подход — **отношения типа HAS-A** (когда один объект содержит другой).

2. Отношения HAS-A («содержит»)

Это связь через владение или использование. Объект одного класса просто содержит ссылку на объект другого. Выделяют три вида таких отношений:

Ассоциация (простая связь) — это самый «свободный» вид связи. Объекты могут взаимодействовать, но они остаются независимыми сущностями. У них разные жизненные циклы, и ни один не является «владельцем» другого.

Пример ассоциации — **Врач и Пациент** или **Преподаватель и Студент**.

Это отношения, при которых объекты разных классов просто взаимодействуют друг с другом. Главное отличие: **ни один объект не является частью другого**. Они существуют сами по себе, но в какой-то момент один объект может использовать методы другого.

Пример: Врач (Doctor) и Пациент (Patient): Врач может лечить пациента, а пациент может прийти к врачу. Но врач не «содержит» пациента внутри себя как часть, и пациент не является частью врача.

Агрегация (слабая связь) — это отношение «целое — часть», при котором часть может существовать **независимо** от целого. Объекты связаны, но их жизненные циклы не переплетены жестко.

Пример: Футбольный клуб (FootballClub) и Игрок (Player): Игрок является частью клуба. Однако, если клуб расформируют (объект `FootballClub` удалится), игрок не исчезнет из системы — он просто станет «свободным агентом» и сможет перейти в другой клуб.

Композиция (строгая связь) — это самая сильная форма связи «целое — часть». В этом случае часть **полностью зависит** от жизненного цикла целого. Она создается вместе с владельцем и уничтожается вместе с ним.

Пример: Здание (Building) и Комната (Room): Комната не имеет смысла без здания. Она создается прямо внутри конструктора здания, и никто извне не может передать готовую «комнату» или забрать её себе.

В Java композиция часто реализуется через **внутренние классы** (inner classes), чтобы еще сильнее подчеркнуть, что `Room` принадлежит только `Building`.

Тип связи	Отношение	Где создается «часть»?	Жизненный цикл	Пример
Ассоциация	«Использует»	Вне класса.	Независимы. Объекты просто «знакомы».	Врач осматривает пациента.
Агрегация	«Является частью»	Вне класса. Передается в конструктор.	Часть может выжить, если «целое» удалено.	Игрок в составе команды.
Композиция	«Владеет частью»	Внутри класса (через <code>new</code>).	Часть погибает вместе с «целым».	Комната внутри здания.

Реализовать консольное приложение,
удовлетворяющее следующим требованиям:

1. Использовать возможности ООП.
2. Каждый класс должен иметь исчерпывающее смысл название и информативный состав (как правило, содержать следующие элементы: скрытые поля, конструкторы с параметрами и без параметров, методы. Методы должны обеспечивать непротиворечивый, полный, минимальный и удобный интерфейс класса. Аргументировать принадлежность к классу каждого создаваемого метода.).
3. Программа должна гарантировать, что объект находится в рабочем состоянии перед выполнением любых операций.
4. При разработке собственных классов необходимо пользоваться абстрагированием от совокупности свойств реальных предметов, и *выбирать только те характеристики и свойства предмета, которые удовлетворяют решаемой задаче.*
5. При инициализации полей класса (хотя бы одного) продумать **использование перечислений**.
6. В программе **должна выполняться проверка всех разработанных элементов класса**.
7. Классы должны быть грамотно разложены в структуре проекта.
8. Приветствуется более детальная проработка предметной области.

Стратегия проектирования и обработки ошибок. При проектировании взаимодействия объектов придерживайтесь следующей иерархии решений:

- **Уровень саморегуляции (L1):** Если объект может исправить ситуацию сам (например, через `if` и подстановку безопасного значения) — он обязан это сделать.
- **Уровень пользовательского интерфейса (L2):** Если ошибка возникла при вводе в меню (например, ввели букву вместо числа), класс-меню должен переспросить пользователя (через цикл `while` и `try-catch` внутри меню), не «роняя» программу.
- **Уровень исключений (L3):** Выбрасывайте `Exception` только если объект бессилен продолжить работу (например, отсутствует критически важный компонент).

Важно: Избыточное использование исключений — это часто признак «ленивого проектирования», который проявляется как:

1. **Перекалывание ответственности:** Разработчик не продумывает восстановление системы (Fail-Safe), а просто сообщает о крахе.
2. **Маскировка плохой инкапсуляции:** Объект не умеет сам проверять свои данные.
3. **Экономия на «проработке сценариев»:** Нежелание продумывать поведение программы в нештатных ситуациях.
4. **Психология «Это не моя проблема»:** Ожидание, что ошибку обработает кто-то другой (тестировщик или пользователь).

Пример на портале показывает **состав** предметной области (какие поля и методы могут быть), но **не диктует архитектуру**. Вы должны **самостоятельно выбрать и реализовать** тип связи между объектами (Композиция или Агрегация).

Должны присутствовать пункты меню в которых можно: создать объект, изменить объект, проверить функционал (реализацию методов) программы.

Индивидуальные задания

0. Создать объект класса Автомобиль, используя классы Колесо, Двигатель. Методы: ехать, заправляться, менять колесо, вывести на консоль марку автомобиля.

1. Создать объект класса Калькулятор, используя классы Экран, Клавиши, Процессор. Методы: сложить, вычесть, вывести на консоль результат вычислений.

2. Создать объект класса Телевизор, используя классы Экран, Динамик, ПультУправления. Методы: включить, переключить Канал, вывести на консоль текущий канал.

3. Создать объект класса Почта, используя классы Письмо, Отправитель, Адресат. Методы: отправить, получить, вывести на консоль содержимое письма.
4. Создать объект класса Гирлянда, используя классы Лампочка, Шнур, БлокПитания. Методы: включить, сменитьРежим, вывести на консоль количество горящих лампочек.
5. Создать объект класса Велотренажер, используя классы Педали, Руль, Дисплей. Методы: начатьТренировку, изменитьНагрузку, вывести на консоль пройденное расстояние.
6. Создать объект класса Компьютер, используя классы Процессор, ОперативнаяПамять, ЖесткийДиск. Методы: включить, выключить, вывести на консоль состояние системы.
7. Создать объект класса Дом, используя классы Стены, Крыша, Окна. Методы: открытьДверь, закрытьДверь, вывести на консоль количество жильцов.
8. Создать объект класса Робот, используя классы Манипулятор, Датчики, Аккумулятор. Методы: двигаться, сканировать, вывести на консоль уровень заряда.
9. Создать объект класса Ноутбук, используя классы Клавиатура, Экран, Процессор. Методы: включить, выключить, вывести на консоль характеристики.
10. Создать объект класса Аквариум, используя классы Вода, Рыбка, Фильтр. Методы: запустить, покормить, вывести на консоль состояние чистоты воды.
11. Создать объект класса Поезд, используя классы Локомотив, Вагон, Колеса. Методы: отправиться, остановиться, вывести на консоль количество вагонов.
12. Создать объект класса Кофемашина, используя классы Вода, Молоко, КофейныеЗерна. Методы: приготовитьКофе, очистить, вывести на консоль количество порций.
13. Создать объект класса Человек, используя классы Сердце, Руки, Ноги. Методы: идти, бежать, вывести на консоль пульс.
14. Создать объект класса ЭлектроннаяКнига, используя классы Дисплей, Память, Батарея. Методы: открытьКнигу, перелистнутьСтраницу, вывести на консоль уровень заряда.
15. Создать объект класса Фонарик, используя классы Корпус, Лампочка, Батарейка. Методы: включитьСвет, выключитьСвет, вывести на консоль яркость.
16. Создать объект класса РоботПылесос, используя классы Двигатель, Датчик, Аккумулятор. Методы: начатьУборку, зарядиться, вывести на консоль карту помещения.
17. Создать объект класса Телефон, используя классы Камера, Микрофон, Аккумулятор. Методы: сделатьЗвонок, сделатьФото, вывести на консоль заряд батареи.
18. Создать объект класса Фотоаппарат, используя классы Объектив, Матрица, Видоискатель. Методы: сделатьСнимок, сменитьРежим, вывести на консоль настройки.
19. Создать объект класса МузыкальныйПлеер, используя классы Динамик, Память, Батарея. Методы: воспроизвестиПесню, поставитьНаПаузу, вывести на консоль список треков.
20. Создать объект класса Торт, используя классы Корж, Крем, Украшение. Методы: разрезать, съестьКусок, вывести на консоль оставшийся вес.
21. Создать объект класса Корабль, используя классы Корпус, Двигатель, Руль. Методы: плыть, броситьЯкорь, вывести на консоль координаты.
22. Создать объект класса Велосипед, используя классы Рама, Колесо, Цепь. Методы: ехать, тормозить, вывести на консоль текущую передачу.
23. Смартфон, используя классы Процессор, Аккумулятор, Дисплей. Методы: включить, запуститьПриложение, вывести на консоль уровень заряда.
24. Создать объект класса Книга, используя классы Обложка, Страница, Закладка. Методы: открыть, перелистнуть, вывести на консоль текст текущей страницы.
25. Создать объект класса Кофемашина, используя классы Бойлер, Насос, КонтейнерДляЗерен. Методы: молотьКофе, варитьЭспрессо, вывести на консоль остаток воды.

26. Создать объект класса Робот-пылесос, используя классы Двигатель, Щетка, ДатчикПрепятствий. Методы: убирать, заряжаться, вывести на консоль заполненность контейнера.

27. Создать объект класса Микроскоп, используя классы Окуляр, Объектив, Подсветка. Методы: фокусировать, сменитьУвеличение, вывести на консоль текущую кратность.

28. Создать объект класса Квадрокоптер, используя классы Рама, Пропеллер, Видеокамера. Методы: взлететь, сделатьФото, вывести на консоль высоту полета.

29. Создать объект класса Фонарик, используя классы Корпус, Светодиод, Кнопка. Методы: включить, переключитьРежим, вывести на консоль яркость луча.

30. Создать объект класса Принтер, используя классы ПечатающаяГоловка, ЛотокДляБумаги, Картридж. Методы: печать, очиститьГоловку, вывести на консоль количество листов.